GoalDocs - Comprehensive Security & Code Review

Review Date: October 5, 2025

Reviewer: Senior Laravel/PHP/DevOps Engineer

Repository: https://github.com/AbrahamOdoi/goaldocs

Executive Summary

This is a comprehensive Laravel document management system with multi-factor authentication, file permissions, workflows, and security features. While the application demonstrates good architectural patterns, **there are several critical security vulnerabilities that must be addressed immediately**.

Overall Grade: C+ (60/100)

- Security: **D** (Critical vulnerabilities found)
- Code Quality: **B** (Good structure, some concerns)
- Architecture: **B+** (Well organized)
- Performance: **B** (Good practices overall)

🚨 CRITICAL SECURITY VULNERABILITIES

1. CRITICAL: Plain Password Stored in Session

Location: `app/Http/Controllers/Auth/RegisterController.php:48`

```php
// Store plain password in session for post-OTP SMS
session(['plain_password' => $request->password]);
```

Risk Level: CRITICAL

Impact: Session hijacking would expose user passwords in plain text.

Fix:

```php
// REMOVE THIS COMPLETELY - DO NOT STORE PASSWORDS IN SESSION
// Instead, send credentials email immediately after registration
// Or generate a temporary setup link
```

2. CRITICAL: Password Sent via SMS in Plain Text

Location: `app/Http/Controllers/Auth/VerificationController.php:187-200`

```php
$plainPassword = session('plain_password');
$message = "Welcome to GoalDocs! Your registration was successful.\nEmail: $email\nPassword: $plainPassword\nPlease ke
// Send SMS
```

Risk Level: CRITICAL

Impact:

- SMS is unencrypted and can be intercepted
- SMS stored in telco databases indefinitely
- Violates security best practices
- Potential compliance violations (GDPR, etc.)

Fix:

```php
// Option 1: Send password reset link instead
$token = Password::createToken($user);
$resetLink = route('password.reset', ['token' => $token]);
$message = "Welcome to GoalDocs! Set your password here: $resetLink";

// Option 2: Email credentials (still not ideal but better than SMS)
Mail::to($user->email)->send(new CredentialsEmail($user, $password));
$message = "Welcome to GoalDocs! Check your email for login instructions.";
```

3. HIGH: Login Excluded from CSRF Protection

Location: `app/Http/Middleware/VerifyCsrfToken.php:15`

```php
protected $except = [
    'login', // ⚠️ This bypasses CSRF protection
];
```

Risk Level: HIGH

Impact: Makes login vulnerable to CSRF attacks.

Fix:

```php
```

```
protected $except = [
    // Remove 'login' from here
    // The custom handle() method already gracefully handles token mismatches
];
```

4. MEDIUM: Session Encryption Disabled by Default

Location: `config/session.php:50`

```
php

'encrypt' => env('SESSION_ENCRYPT', false),
```

Risk Level: MEDIUM

Impact: Session data stored in plain text, especially concerning given password storage issue.

Fix:

```
php

'encrypt' => env('SESSION_ENCRYPT', true),
```

Update `.env`:

```
SESSION_ENCRYPT=true
```

⚠️ SECURITY CONCERNS

5. OTP Stored in Plain Text

Location: `app/Http/Controllers/Auth/VerificationController.php:42-51`

OTP codes are stored without hashing. While short-lived, this is still a concern.

Recommendation:

```
php
```

```php
// Store hashed OTP
DB::table('otp_codes')->updateOrInsert(
    ['user_id' => $user->id],
    [
        'code' => hash('sha256', $code), // Hash the code
        'channel' => $request->channel,
        'expires_at' => now()->addMinutes(15),
        'created_at' => now(),
        'updated_at' => now(),
    ]
);

// When verifying:
$otp = DB::table('otp_codes')
    ->where('user_id', $user->id)
    ->where('code', hash('sha256', $request->code)) // Hash for comparison
    ->where('expires_at', '>', now())
    ->first();
```

6. SMS API Credentials in Config

Location: `config/services.php` (not in uploaded files)

Ensure SMS credentials are in `.env`, not hardcoded:

```php
'deywuro_sms' => [
    'username' => env('DEYWURO_SMS_USERNAME'),
    'password' => env('DEYWURO_SMS_PASSWORD'),
    'source' => env('DEYWURO_SMS_SOURCE'),
    'url' => env('DEYWURO_SMS_URL'),
],
```

7. All New Users Are Admins by Default

Location: `app/Http/Controllers/Auth/RegisterController.php:40`

```php
'is_admin' => true, // All new signups are admin by default
```

Risk: First user should be admin, subsequent users should not be.

Fix:

```php
```

```
'is_admin' => User::count() === 0, // Only first user is admin
```

✅ SECURITY POSITIVES

Good Practices Found:

1. **File Upload Security**
   - Blacklist of dangerous extensions
   - File size limits (100MB)
   - Good MIME type validation
2. **Rate Limiting**
   - OTP requests limited to 5 per hour
   - Prevents brute force attacks
3. **Password Hashing**
   - Using Laravel's Hash facade (bcrypt)
   - Proper password verification
4. **MFA Implementation**
   - Two-factor authentication via OTP
   - Multiple channels (email/SMS)
5. **Authorization Structure**
   - File permissions system
   - Department/position hierarchy
   - Role-based access control
6. **Session Security**
   - HttpOnly cookies enabled
   - SameSite set to 'lax'
   - Database session driver

🏗️ CODE QUALITY REVIEW

Strengths:

1. **Well-Organized Structure**
   - Clear separation of concerns
   - Proper use of Laravel conventions
   - Good controller organization

2. **Model Relationships**
   - Well-defined Eloquent relationships
   - Proper use of pivot tables
   - Good accessor methods
3. **Database Design**
   - Comprehensive migrations
   - Proper foreign keys
   - Audit trail support
4. **Feature-Rich**
   - Document management
   - Version control
   - Workflows
   - Security monitoring
   - Analytics

Areas for Improvement:

1. **Fat Controllers**
   - FileController is 1424 lines (too large)
   - Should use Service classes
   - Business logic should be in services

Recommendation:

```
php

// Create services:
app/Services/FileService.php
app/Services/PermissionService.php (already exists)
app/Services/VersionControlService.php
```

2. **Missing Validation Rules** Some controllers have inline validation. Consider:

```
php

// Create Form Requests
app/Http/Requests/FileUploadRequest.php
app/Http/Requests/UserRegistrationRequest.php
```

3. **Error Handling** Add global exception handling:

```
php
```

```php
// app/Exceptions/Handler.php
public function render($request, Throwable $exception)
{
    if ($exception instanceof ModelNotFoundException) {
        return response()->json(['error' => 'Resource not found'], 404);
    }
    return parent::render($request, $exception);
}
```

4. **Logging** Inconsistent logging approach. Standardize:

```php
// Use consistent logging
Log::channel('security')->warning('Failed login attempt', [
    'email' => $request->email,
    'ip' => $request->ip()
]);
```

🚀 PERFORMANCE RECOMMENDATIONS

Database:

1. **Add Indexes**

```php
// Add to migrations
$table->index(['user_id', 'created_at']); // file_permissions
$table->index('expires_at'); // otp_codes
$table->index(['type', 'type_name']); // folders, files
```

2. **Eager Loading** Prevent N+1 queries:

```php
// In FileController
$files = File::with(['uploader', 'permissions', 'versions'])
    ->where('folder_id', $folderId)
    ->get();
```

3. **Caching**

```php
```

```php
// Cache frequently accessed data
$departments = Cache::remember('departments_' . $user->type, 3600, function() {
    return Department::all();
});
```

File Storage:

1. **Use CDN for Static Assets**
   - Configure CloudFront/CloudFlare
   - Serve uploaded files through CDN
2. **Queue Large Operations**

```php
// Dispatch jobs for heavy operations
ProcessFileUpload::dispatch($file);
GenerateThumbnail::dispatch($file);
```

📝 IMMEDIATE ACTION ITEMS

Priority 1 (CRITICAL - Fix Within 24 Hours):

1. **Remove password from session storage**
   - Delete line 48 in RegisterController.php
   - Remove password SMS functionality
   - Implement password reset link instead
2. **Stop sending passwords via SMS**
   - Implement secure alternative
   - Use email with password reset link
3. **Remove 'login' from CSRF exceptions**
   - Delete line 15 in VerifyCsrfToken.php

Priority 2 (HIGH - Fix Within 1 Week):

4. Enable session encryption
5. Hash OTP codes before storage
6. Fix admin default for new users
7. Review and test all authentication flows

Priority 3 (MEDIUM - Fix Within 2 Weeks):

8. Refactor FileController into services

9. Create Form Request classes
10. Add database indexes
11. Implement eager loading
12. Set up proper logging

🔒 SECURITY CHECKLIST

- [ ] Remove plain password storage
- [ ] Stop SMS password transmission
- [ ] Enable CSRF for login
- [ ] Enable session encryption
- [ ] Hash OTP codes
- [ ] Fix admin defaults
- [ ] Add rate limiting to login
- [ ] Implement account lockout after failed attempts
- [ ] Add security headers (HSTS, CSP, X-Frame-Options)
- [ ] Set up security monitoring
- [ ] Configure proper CORS
- [ ] Use HTTPS only in production
- [ ] Regular security audits
- [ ] Dependency updates (composer update)

🧪 TESTING RECOMMENDATIONS

1. **Write Tests**

```
bash

php artisan make:test Auth/LoginTest
php artisan make:test Auth/RegistrationTest
php artisan make:test FileUploadTest
```

2. **Security Testing**
   - Penetration testing
   - OWASP Top 10 checks
   - SQL injection testing
   - XSS testing
3. **Load Testing**
   - Test file upload limits
   - Concurrent user testing

- Database performance

📊 COMPLIANCE CONSIDERATIONS

GDPR Compliance:

1. **Data Minimization**
   - Don't store passwords in session
   - Don't send passwords via SMS
   - Implement data retention policies
2. **User Rights**
   - Right to be forgotten
   - Data portability
   - Consent management
3. **Audit Trails**
   - Log all access to sensitive data
   - Track file access
   - Monitor authentication events

🎯 RECOMMENDATIONS SUMMARY

Security (Must Do):

1. Remove password session storage
2. Stop SMS password transmission
3. Enable CSRF for login
4. Enable session encryption
5. Hash OTP codes

Code Quality (Should Do):

1. Refactor large controllers
2. Create service classes
3. Add Form Requests
4. Improve error handling
5. Standardize logging

Performance (Nice to Have):

1. Add database indexes

2. Implement caching
3. Use CDN
4. Queue heavy operations
5. Optimize queries

📞 NEXT STEPS

1. **Immediate:** Fix critical security vulnerabilities
2. **This Week:** Address high-priority security concerns
3. **This Month:** Code quality improvements
4. **Ongoing:** Performance optimization

📄 CONCLUSION

GoalDocs is a feature-rich document management system with good architectural foundation. However, **critical security vulnerabilities must be addressed immediately** before this application can be safely used in production.

The password handling issues (session storage and SMS transmission) are particularly concerning and should be your top priority. Once these are fixed, the application has solid potential.

Recommended Timeline:

- **Day 1-2:** Fix critical security issues
- **Week 1:** Address high-priority concerns
- **Week 2-4:** Code quality improvements
- **Month 2:** Performance optimization

End of Report

This review was conducted as a comprehensive security and code quality assessment. For questions or clarifications, please discuss specific sections.